

Übersetzung und Analyse von Programmen

The background is a stylized, low-poly illustration. It features a multi-story building with large windows in shades of blue and green. A large, bright yellow sun is in the upper right corner. In the foreground, there are two green trees with yellow leaves and a large, white flower with a yellow center. The overall color palette is dominated by blues, greens, and yellows.

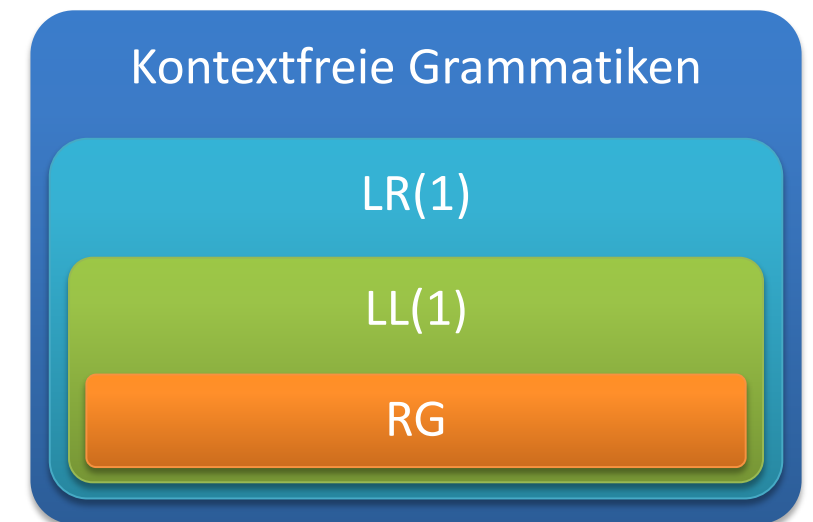
Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl

Informatik, Universität Trier

PROJEKT II – LEXER & PARSER

- PLY Kurzform für Python Lex-Yacc
- Kombiniert die Erzeugung von Lexer und Parser.
- In C oder Java werden meist zwei getrennte Werkzeuge (lex und yacc bzw. flex und cup) verwendet.
- Erzeugt aus regulären Ausdrücken einen DEA, der auch als Scanner oder Lexer bezeichnet wird.
- Erzeugt LALR(1)-Parser
 - Benötigt weniger Speicher als ein LR(1) Parser, aber die Menge der durch eine LALR(1)-Grammatik beschreibbaren Sprachen ist kleiner.
 - $LR(1) > LALR(1) > LR(0)$
 - Berechnet Bottom-Up eine Rechtsableitung
- Dokumentation:
 - <https://www.dabeaz.com/ply/ply.html>



- Paket PLY in Python installieren
 - In PyCharms File->Settings->Project->Python Interpreter mit + Paket hinzufügen)
- Lexer und Parser in zwei getrennten Python-Dateien definieren, die `ply.yacc` bzw. `ply.lex` importieren
- Der erzeugte Parser bietet die Methode `parse(text)`

Angabe von regulären Ausdrücken für jedes reservierte Wort ist nicht notwendig, wenn man diese z.B. in der Regel für Bezeichner als Sonderfall herausfiltert:

```
def t_ID(t):  
    r'[a-zA-Z_][a-zA-Z_0-9]*'  
    # Check for reserved words  
    t.type = reserved.get(t.value, 'ID')  
    return t
```

```
import ply.lex as lex  
  
reserved = { 'while': 'WHILE', 'do': 'DO' }  
  
# List of token names. This is always required  
tokens = [ 'CONST', 'LBRACE', 'RBRACE' ] +  
list(reserved.values())  
  
# Regular expression rules for simple tokens  
t_LBRACE = r'\{'  
t_RBRACE = r'\}'  
t_WHILE = r'while'  
t_DO = r'do'  
  
# A regular expression rule with some action code  
def t_CONST(t):  
    r'd+'  
    t.value = int(t.value)  
    return t  
  
# Define a rule so we can track line numbers  
def t_newline(t):  
    r'\n+'  
    t.lexer.lineno += len(t.value)  
  
# A string containing ignored characters (spaces and tabs)  
t_ignore = ' \t'  
  
# Error handling rule  
def t_error(t):  
    print("Illegal character '%s'" % t.value[0])  
    t.lexer.skip(1)  
  
# Build the lexer  
lexer = lex.lex()
```

Im Gegensatz zu LL-Grammatiken sind LR-Grammatiken nicht rechtsrekursiv.

Vorgehensweise:
Mehrdeutigkeiten und
Rechtsrekursion entfernen.

parser.out
Informationen zum Debuggen

parstab.py
Pythoncode der Parsertabelle

```
import ply.yacc as yacc
import syntax as ast

# Get the token map from the lexer. This is required.
from triplalex import tokens

precedence = (
    )

def p_expression_const(p):
    'expression : CONST'
    p[0] = ast.CONST(p[1])

def p_expression_while(p):
    'expression : WHILE expression DO LBRACE expression RBRACE'
    p[0] = ast.WHILE(p[2],p[5])

#def p_empty(p):
#    'empty :'
#    pass

# Error rule for syntax errors
def p_error(p):
    print("Syntax error in input!")

# Build the parser
parser = yacc.yacc() # debug=True
```

Präzedenz und Assoziativität

Man kann Nichtterminalen eine Präzedenz geben, diese wird dann bei Grammatiken, **die nicht LALR(1) sind**, bei der Auflösung von Konflikten bei der Generierung berücksichtigt.

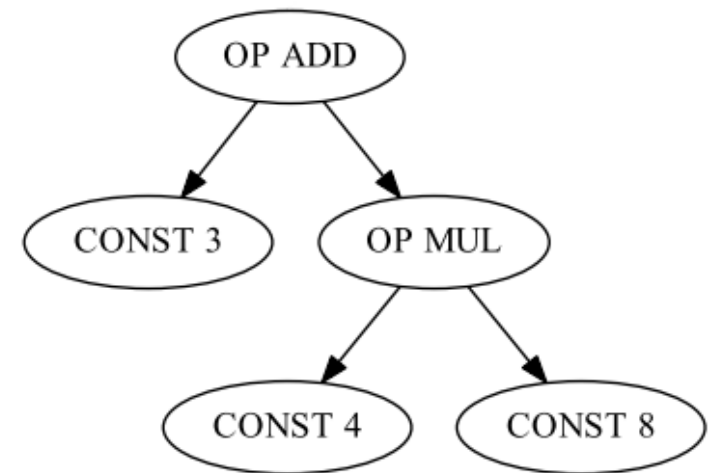
Assoziativität: left, right, nonassoc

```
precedence = (  
    ('left', 'PLUS', 'MINUS'),  
    ('left', 'MUL', 'DIV')  
)
```

niedrig



hoch



Löst **Shift Reduce Konflikte** auf

Beispiel: $3 + 4 * 8$

Ohne Präzedenz ist unklar, ob mit Regel 1 reduziert oder zuerst mit Regel 2 geshiftet, d.h. das nächste Zeichen „erkannt“ (shift) werden soll.

1: $E \rightarrow E + E$

2: $E \rightarrow E * E$

3: $E \rightarrow \text{int}$

Präzedenz und Assoziativität

1: $E \rightarrow E + E$

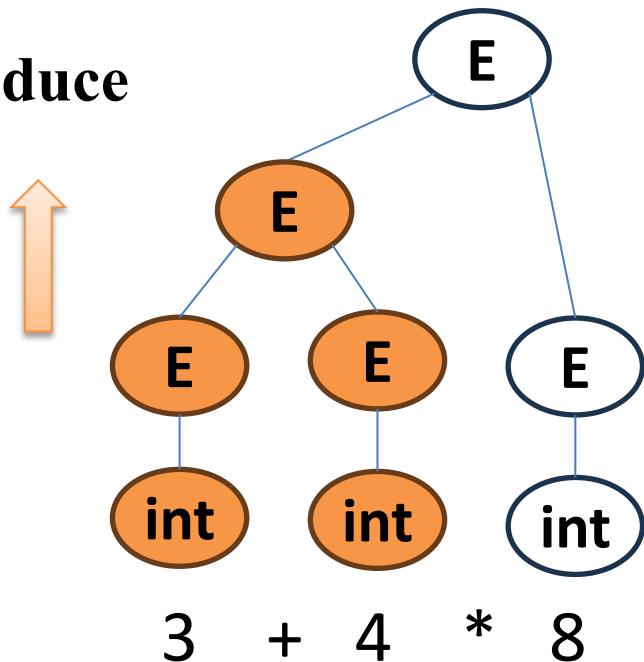
2: $E \rightarrow E * E$

3: $E \rightarrow \text{int}$

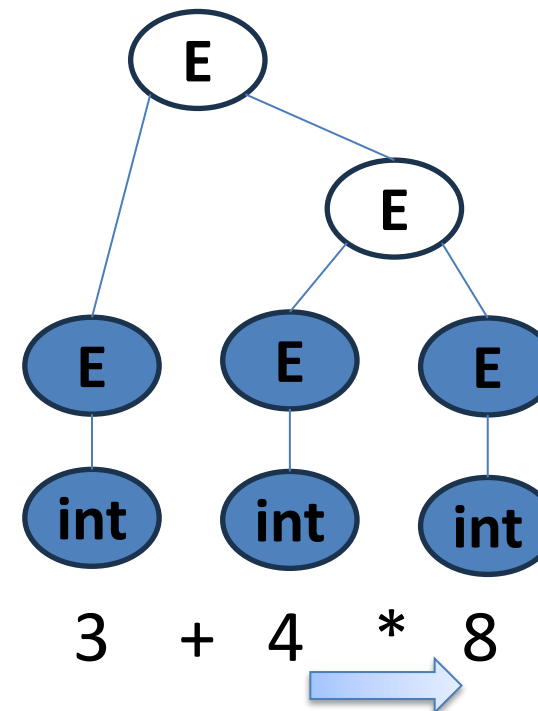
$E \xRightarrow{rm} E + E \xRightarrow{rm} E + E * E \xRightarrow{rm} E + E * \text{int} \xRightarrow{rm} E + \text{int} * \text{int} \xRightarrow{rm} \text{int} + \text{int} * \text{int}$

$E \xRightarrow{rm} E * E \xRightarrow{rm} E * \text{int} \xRightarrow{rm} E + E * \text{int} \xRightarrow{rm} E + \text{int} * \text{int} \xRightarrow{rm} \text{int} + \text{int} * \text{int}$

reduce



shift



Weitere Details:

- Alle Terminale ohne explizite Präzedenz/Assoziativität haben niedrigste Präzedenz.
- Assoziativität wird als verwendet um Shift-Reduce Konflikte aufzulösen. Allerdings nur bei gleicher Präzedenz.
- Folgen zwei Terminale, die mit **nonassoc** deklariert wurden, aufeinander und haben gleiche Präzedenz, dann wird eine Fehlermeldung ausgegeben.
 - Zum Beispiel sollte der Parser für die Eingabe $6 == 7 == 8 == 9$ einen Fehler erzeugen, wenn '=' als nonassoc deklariert wurde.